```
Q1------slip4----------------------------------------
import heapq
GOAL = (1, 2, 3, 0)
def heuristic(state):
    distance = 0
    for i in range(4):
        if state[i] != 0:
            goal_position = GOAL.index(state[i])
            current_row = i // 2
            current_col = i % 2
            goal_row = goal_position // 2
            goal_col = goal_position % 2
            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)
    return distance
def get_neighbors(state):
    neighbors = []
    blank = state.index(0)
    row = blank // 2
    col = blank % 2
    moves = [(-1, 0),(1, 0),(0, -1),(0, 1)]
    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc
        if 0 <= new_row < 2 and 0 <= new_col < 2:
            new_blank = new_row * 2 + new_col
            new_state = list(state)
            new_state[blank], new_state[new_blank] = \
                new_state[new_blank], new_state[blank]
            neighbors.append(tuple(new_state))
    return neighbors
def a_star(start):
    priority_queue = []
    heapq.heappush(
        priority_queue,
        (heuristic(start), 0, start, [])
    )
    visited = set()
    while priority_queue:
        f, g, state, path = heapq.heappop(priority_queue)
        if state in visited:
            continue
        visited.add(state)
        if state == GOAL:
            return path + [state]
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                new_g = g + 1
```

```
            new_f = new_g + heuristic(neighbor)
            heapq.heappush(
               priority_queue,
               (
                  new_f,new_g,neighbor,
                  path + [state]
               )
            )
   return None
def display(state):
   print(state[0],state[1])
   print(state[2],state[3])
start = (1, 0, 3, 2)
print("4-Puzzle Problem using A* Search")
print("\nInitial State:")
display(start)
solution = a_star(start)
if solution:
   print("\nSolution Path:")
   print("Number of moves:", len(solution) - 1)
   for i, state in enumerate(solution):
      print("\nStep", i)
      display(state)
else:
   print("\nNo solution exists.")
```

```
Q2-----slip4----------------------------------------
tree = [[3, 5], [6, 9]]
def minimax(node, depth, is_max):
   if depth == 2:
      return node
   if is_max:
      return max(minimax(x, depth+1, False)
            for x in node)
   return min(minimax(x, depth+1, True)
         for x in node)
def alphabeta(node, depth, alpha, beta, is_max):
   if depth == 2:
      return node
   if is_max:
      value = float('-inf')
      for x in node:
         value = max(value, alphabeta(
            x, depth+1, alpha, beta, False))
         alpha = max(alpha, value)
         if alpha >= beta:
            break
      return value
   else:
      value = float('inf')
      for x in node:
         value = min(value, alphabeta(
            x, depth+1, alpha, beta, True))
         beta = min(beta, value)
         if alpha >= beta:
            break
      return value
print("Minimax:", minimax(tree, 0, True))
print("Alpha-Beta:", alphabeta(
   tree, 0, float('-inf'), float('inf'), True))
```

Slip3---q1----

```
import java.util.*;

class DFS {
   static Map<Character, List<Character>> graph = new HashMap<>();
   static Set<Character> visited = new HashSet<>();

   static void dfs(char node) {
      visited.add(node);
      System.out.print(node + " ");
      for (char next : graph.get(node)) {
         if (!visited.contains(next)) {
            dfs(next);
         }
      }
   }

   public static void main(String[] args) {
      graph.put('A', Arrays.asList('B', 'C', 'E'));
      graph.put('B', Arrays.asList('A', 'D'));
      graph.put('C', Arrays.asList('A', 'F'));
      graph.put('D', Arrays.asList('B', 'E'));
      graph.put('E', Arrays.asList('D', 'F', 'A'));
 graph.put('F', Arrays.asList('C', 'E'));

      System.out.print("DFS Traversal: ");
      dfs('A');
   }
}
```

**Slip 3------q2-------**

```
from sklearn.svm import SVC
# Training data
X = [
 [1, 2],
 [2, 3],
 [5, 5]
]
# Class labels
y = [0, 0, 1]
# Test input
X_test = [[4, 4]]
# Create Non-Linear SVM using RBF kernel
model = SVC(kernel='rbf')
# Train the model
model.fit(X, y)
# Predict the class of test input
prediction = model.predict(X_test)
print("Training Data:")
print(X)
print("\nClass Labels:")
print(y)
print("\nTest Input:")
print(X_test)
print("\nPredicted Class:")
print(prediction)
```

Q1------slip4----------

```
import heapq
GOAL = (1, 2, 3, 0)
def heuristic(state):
    distance = 0
    for i in range(4):
        if state[i] != 0:
            goal_position = GOAL.index(state[i])
            current_row = i // 2
            current_col = i % 2
            goal_row = goal_position // 2
            goal_col = goal_position % 2
            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)
    return distance
def get_neighbors(state):
    neighbors = []
    blank = state.index(0)
    row = blank // 2
    col = blank % 2
    moves = [(-1, 0),(1, 0),(0, -1),(0, 1)]
    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc
        if 0 <= new_row < 2 and 0 <= new_col < 2:
            new_blank = new_row * 2 + new_col
            new_state = list(state)
            new_state[blank], new_state[new_blank] = \
                new_state[new_blank], new_state[blank]
            neighbors.append(tuple(new_state))
    return neighbors
def a_star(start):
    priority_queue = []
    heapq.heappush(
        priority_queue,
        (heuristic(start), 0, start, [])
    )
    visited = set()
    while priority_queue:
        f, g, state, path = heapq.heappop(priority_queue)
        if state in visited:
            continue
        visited.add(state)
        if state == GOAL:
            return path + [state]
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                new_g = g + 1
```

```
            new_f = new_g + heuristic(neighbor)
            heapq.heappush(
               priority_queue,
               (
                  new_f,new_g,neighbor,
                  path + [state]
               )
            )
   return None
def display(state):
   print(state[0],state[1])
   print(state[2],state[3])
start = (1, 0, 3, 2)
print("4-Puzzle Problem using A* Search")
print("\nInitial State:")
display(start)
solution = a_star(start)
if solution:
   print("\nSolution Path:")
   print("Number of moves:", len(solution) - 1)
   for i, state in enumerate(solution):
      print("\nStep", i)
      display(state)
else:
   print("\nNo solution exists.")
```

```
Q2-----slip4----------------------------------------
tree = [[3, 5], [6, 9]]
def minimax(node, depth, is_max):
   if depth == 2:
      return node
   if is_max:
      return max(minimax(x, depth+1, False)
            for x in node)
   return min(minimax(x, depth+1, True)
         for x in node)
def alphabeta(node, depth, alpha, beta, is_max):
   if depth == 2:
      return node
   if is_max:
      value = float('-inf')
      for x in node:
         value = max(value, alphabeta(
            x, depth+1, alpha, beta, False))
         alpha = max(alpha, value)
         if alpha >= beta:
            break
      return value
   else:
      value = float('inf')
      for x in node:
         value = min(value, alphabeta(
            x, depth+1, alpha, beta, True))
         beta = min(beta, value)
         if alpha >= beta:
            break
      return value
print("Minimax:", minimax(tree, 0, True))
print("Alpha-Beta:", alphabeta(
   tree, 0, float('-inf'), float('inf'), True))
```